

Chapter:-1 Introduction and Installation
Chapter:-2 Application deployment via command line.
Chapter:-3 Pod Sheduling
Chapter:-4 Volume
Chapter:-5 Networking
Chapter:-6 Resource and compute
Chapter:-7 Security
Chapter:-8 Monitoring

Basics commands

=====

```
kubectl get nodes
kubectl get pod
kubectl get pod -A
kubectl get pod -n kube-system
kubectl get pods --all-namespaces
kubeadm token create --print-join-command
kubeadm token list
kubectl describe pod test
kubectl describe node kube-master
kubectl delete pod test
kubectl delete pod --all
kubectl run test --image=docker.io/nginx
kubectl exec -it test bash
kubectl create namespace myns --dry-run=client -o yaml
```

```
kubectl get pod --show-labels
kubectl get pod -l color=red
kubectl label pod test team=openstack
kubectl run app --image=nginx -l color=sky
kubectl label node kube-node1 flavor=redhat
kubectl get nodes -l flavor=redhat
```

```
kubeadm config print init-defaults
kubectl get events
kubectl get ep demo
```

```
kubectl expose deployment nginx --type=LoadBalancer
kubectl scale deployment nginx --replicas=0
kubectl config view
```

Software Development Life Cycle.

1. Deployment phase
2. Testing phase
3. Operation phase
4. Production phase

Docker Container

=====

Application => DockerFile => docker image => container

Problems of docker

=====

1. Security
2. Upgradation
3. Migration
4. Autoscaling
5. Downtime

Container Orchestration System(Tools)

1. kubernetes
2. Docker Swarm
3. Apache Mesos

4. Dokku

Chapter1 : Introduction and Installation

=====

Q) What is kubernetes?

=> Kubernetes is a open-source orchastration and management tools for containers. (it's give the instruction to components to work together)

=> Developed by Google and written in Golang.

=> Containerized Orchestration System.

=> You can manage and do work like upgrade, manager, configure, scaling of containers from central location.

=> Provide Application Deployment, Scaling and Management on demand.

Some other paid K8s cluster are also avabile:-

=====

a. EKS: Amazon Elastic K8s service. (readymate kubernetes cluster manager by AWS)

b. GKE: Google k8s engine.

c. AKS: Azure k8s service.

d. microk8s: k8s for development.(free for local testing)

Note: All cloud have different platform but kubernetes functions are same.

Q) Advantages or Why we need K8s?

1. Centerilized management

=> We can manage pods(containers) from central server(control-plane) and can do daily tasks like pod creation, deletion, upgradation, deployment, scalling of pods. etc.

2. Cluster (Failover).

=> If any workder node went down due to any reason, in that case all workload of (containers) will be failover(migrate) to on another node.

3. Scalability: Increased containers numbers for running application as per demand.

Scalability are two types(kubernetes support both types)

a) Manually

b) Automatically(you can set threshold like 85%)

To perform scalability we have two mode.

a) Vertical form => To inscrease the resources like cpu, memory of existing containers.

b) Horizontal form => when you nedd to increase instances count of containers.

4. Rollout and rollback.

=> In kubernetes you can upgrade your application with zero downtime.

=> rollback means you can back to old version if lastet version not compatable with application.

6. Auto-mate Docker Deployment :- K8s can help you to manage automate the docker deployment.

7. Auto-Healing : If a container is not healthy then K8s can auto replace, re-deploy, restart the container. It is now K8s responsibility.

Q) What is Statefulset?

=> Statefulset are kubernetes resource that allow us to deploy and manage

stateful applications.

Stateful Applications : Applications that required persistence storage to run applications. like (Database applications)

Stateless Applications : Applications which do not require pv storage to run applications. (web server)

Q) Explain kubernetes Arch?

1. kubernetes cluster:

=> When set of docker machines works together to manage application managed by kubernetes server it's called one cluster.

=> You can create cluster with Physical machines or virtual machines.

=> Virtual (it's depend on company what they are using). It could be. => Kvm/vmware/microsofts or cloud based machines.

2. kubernetes Master: A machine from where we can manage workerNodes.

3. Worker Nodes : Where the pod containers are runs.

Note:

Pod

=> Smallest Unit of Kubernetes. where we deploy our applications. pod is like a rapper which contains containers resources like network, storage.

=> We can deploy more than one container inside the pod.

Interview Question:

=====

Q) What are the services running on master node and worker node?

Total we have 6 components in k8s.

1. Kube Api-server (Primary Management component)

2. Kube Scheduler:

3. Kube Controller manager:

4. ETCD:

5. kublet:

6. kube proxy:

There are 4 services which running on master node.

1) kube-api: api means listen:

=> It does validate (authentication and authorization) of any instructions which giving by kubernetes administrator.

(means check the roles and user rights, do they have right to execute the command and working area of user.)

=> It is responsible to perform every action within the cluster.

=> if api went down then existing workload would be down.

2) kube-etcd

=> It is the k8s database which contains api related information or contains complete cluster information. like Deployment information, network Information.

=> No one can directly access the etcd database except kube-api server.(Only kube-api server can access etcd directly)

=> NoSQL DB

=> key Value Form

=> Storage information about cluster.

3) kube-scheduler:

=> It check the health of all worker node first then decide which one is best for your application.

=> It decides the right worker node for pod containers.

Two type of scheduling in scheduler

a) default : ?

b) custom :

52.16 min (videos)

4) kube-controller:

- => Its works like a brain of k8s.
- => Provide api instructions to worker node.
- => it give handover api command to worker node.
- => Maintain communication with worker node through API.

There are some controller in K8s

1. Node Controller: Monitor the status of Nodes and take necessary actions if anything happend like node failur

Maintain desire number of nodes in k8s.

2. Replication Controller : Monitor the Status of Pods and take necessary actions.

Types of contollers:

- a) RC
- b) RS
- c) Deployment

Command flow (55:00) videos

command(kubecttl)=> kube-apiserver <== etcd => scheduler => controller => workderNodes => kublete (dockerd/containerd/cri-io) any socket.

Q) What are the services of worker nodes:

1) Container Socket:

- => dockerd: (Not in use now days)
- => containerd:

Redhat/IBM annous new socket:

=> cri-io

2) kubelet:

- => Its a agent of kubernetes master which need to run on each worker node. without kubelet kubernetes cannot control container host.
- => It take instruction from kubernetes master and execute on worker node.
- => It is responsible for all the activity on worker nodes.

Flow(when kube-api request kubelet to create pod)

=====

kube-api (command/request to create container) => kubelet (request to container runtime engine(containerd) to create container) => containerd create pod.

containerd(Send status of pod has been created) => kubelet(forwad this information to) => kube-api(forward to etcd) => etcd(store status in db)

3) kube-proxy:

- => Responsible for communication between pods.
- => Manage your container traffic layer work.

Kubernetes Setup:

- 1) User managed
- 2) k8s cluster setup on cloud.
 - a) AWS: EKS (ready kubernetes cluster manager by AWS)
 - b) Google: GKE
 - c) Azure: AKS

In Production

- => Multi-Master node cluster
- => Multiple worker node

=====

Installation Requirements are:

1) Cloud Based:

- a) Master Node: OS=> Ubuntu => 2G memory => 1 CPU/ 10HDD
- b) worker Node: 2GB /1 CPU/ 10HDD: Ubuntu

Installation method:

- 1) Systemd Mode: as a service mode.
- 2) control plane: all components running in separate containers. OS layer will be unchanged.

Kubeadm: installation utility

kubectl: command utility (only required master nodes)

kubelet: agent of kubernetes master

Chapter:-2 Application deployment via command line.

=====

We have three ways to interact with API

- 1) command method (Imperative way) : (kubectl is a tool to manage containers). you cannot perform 100% percent work via command line.
- 2) file method (Declarative way : (write instruction in file). You can perform 100% percent work via command line.
- 3) GUI mode: :

By default not installed. you can install GUI 3rd party tools.

Note:

- => By default we deploy services containers in kubernetes.
- => We can also deploy raw containers, but only when we need to do change on images.
- => But it has a different way. Sir, will explain later.

```
root@kube-master:~# cat /etc/kubernetes/admin.conf | grep server
server: https://192.168.122.241:6443 <== API
port
root@kube-master:~# hostname -i
192.168.122.241
root@kube-master:~# cat /etc/kubernetes/admin.conf | grep user
user: kubernetes-admin <== User

root@kube-master:~# netstat -tulnp| grep api
tcp6      0      0 :::6443          :::*              LISTEN
1182/kube-apiserver
root@kube-master:~# netstat -tulnp| grep contr
tcp        0      0 127.0.0.1:10257  0.0.0.0:*          LISTEN
1129/kube-controller
root@kube-master:~# netstat -tulnp| grep kubelet
tcp        0      0 127.0.0.1:10248  0.0.0.0:*          LISTEN
690/kubelet
tcp6      0      0 :::10250         :::*              LISTEN
690/kubelet
```

Note: Scheduler cannot control kube-apiserver, etcd, controller. these four services or component are hardcoded on the master node. and never moved on worker node.

Scheduler is not required if we scheduled pod manually.

IQ)

Q) How to scheduler decide which node is healthy?

It depends on four things (In simple it depends on predicates)

- a) CPU pressure
- b) Memory pressure
- c) numbers of containers

d) how many ports are available.

You can check this via ==> "root@kube-master:~# kubectl describe node kube-node1"

Q) Why we set three master node? ==> the number should always odd.
Can you have multiple master nodes in Kubernetes.

Session : 3 (Now start file method)

=====

Note:

=> manifest keywords are pre-defined.

=> we define values in key=value in yaml file.

=> keywords will change according to kind.(indentation should be fine)

=> Details keywords of manifest(this come in CKED)

Details about file method for yaml (Type of key in manifest)

1) string (key=value)

2) list

3) mapping (object)
meta

Understanding of manifest

=====

root@kube-master:~# cat > test.yaml

apiVersion: v1

kind: Pod

<== Kind is Instruction

type (it is case-sensitive)

creationTimestamp: null

labels:

run: test

name: test

spec:

containers:

- image: docker.io/nginx

name: test

resources: {}

dnsPolicy: ClusterFirst

restartPolicy: Always

status: {}

root@kube-master:~# kubectl api-resources

<== To list all type of kinds

root@kube-master:~# kubectl api-resources --help

root@kube-master:~# kubectl api-resources -o wide

root@kube-master:~# kubectl api-resources --namespaced=false ==> resource

will for all the cluster, not for specific project line pv.

root@kube-master:~# kubectl api-resources --namespaced=true ==> pod will

only for specific project.

root@kube-master:~# kubectl explain pod

root@kube-master:~# kubectl explain pod.metadata

<== list keywords in metadata.

root@kube-master:~# kubectl explain pod --recursive=true

root@kube-master:~# kubectl explain pod.spec.containers.resources

root@kube-master:~# kubectl explain pod.spec.containers.ports

=====ETCD database=====

root@kube-master:~# kubectl describe pod etcd-kube-master -n kube-system (etcd store data in external data)

Volumes:

etcd-certs:

```
Type:          HostPath (bare host directory volume)
Path:          /etc/kubernetes/pki/etcd
HostPathType:  DirectoryOrCreate
etcd-data:
  Type:        HostPath (bare host directory volume)
  Path:        /var/lib/etcd
               <=== external storage from base machine
  HostPathType: DirectoryOrCreate
QoS Class:     Burstable
Node-Selectors: <none>
Tolerations:   :NoExecute op=Exists
Events:        <none>
```

Q) How to create services in k8s?

1. ClusterIP (default service)
2. NodePort
3. Loadbalancer
4. ExternalName

```
# kubectl expose pod red --port=80 --dry-run=client -o yaml
# kubectl expose pod red --name=red-service --port=80 --dry-run=client -o
yaml
# kubectl expose pod red --name=red-service --type=NodePort --port=80 --dry-
run=client -o yaml
# kubectl get svc
```